Operating Systems

Dmitry Zaitsev

Lecture 19: Memory management in OS. Virtual memory. Shared segments.

Historical memory allocation methods

Basic principles:

1) contiguous memory of a process
2) entire process should be present in RAM to run it

Memory control approaches:

- Single partition
- Fixed-size partitions
- Dynamic partitions

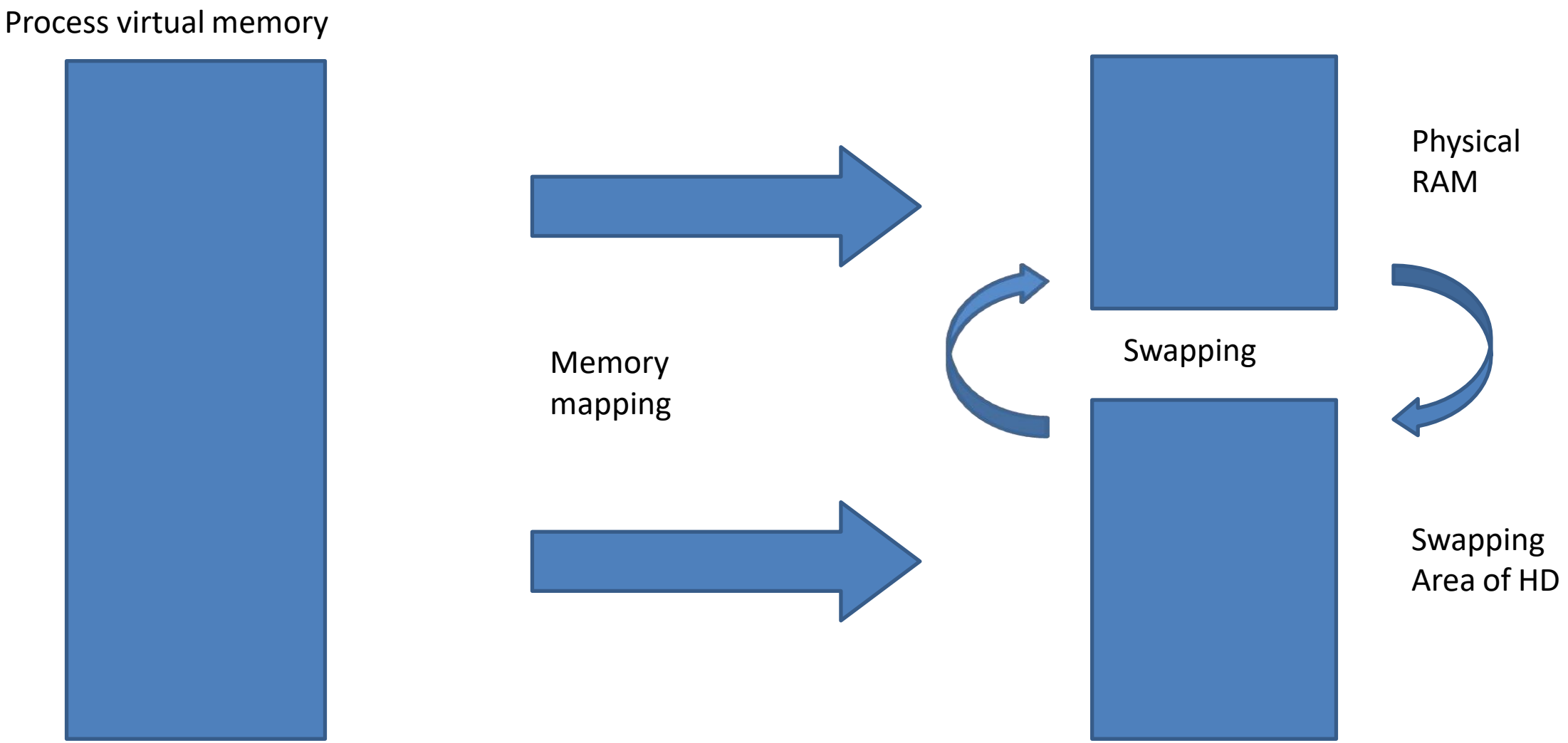
Problems: memory fragmentation, memory protection, lack of memory

Solutions: relocation of memory, limit registers, overlay structures

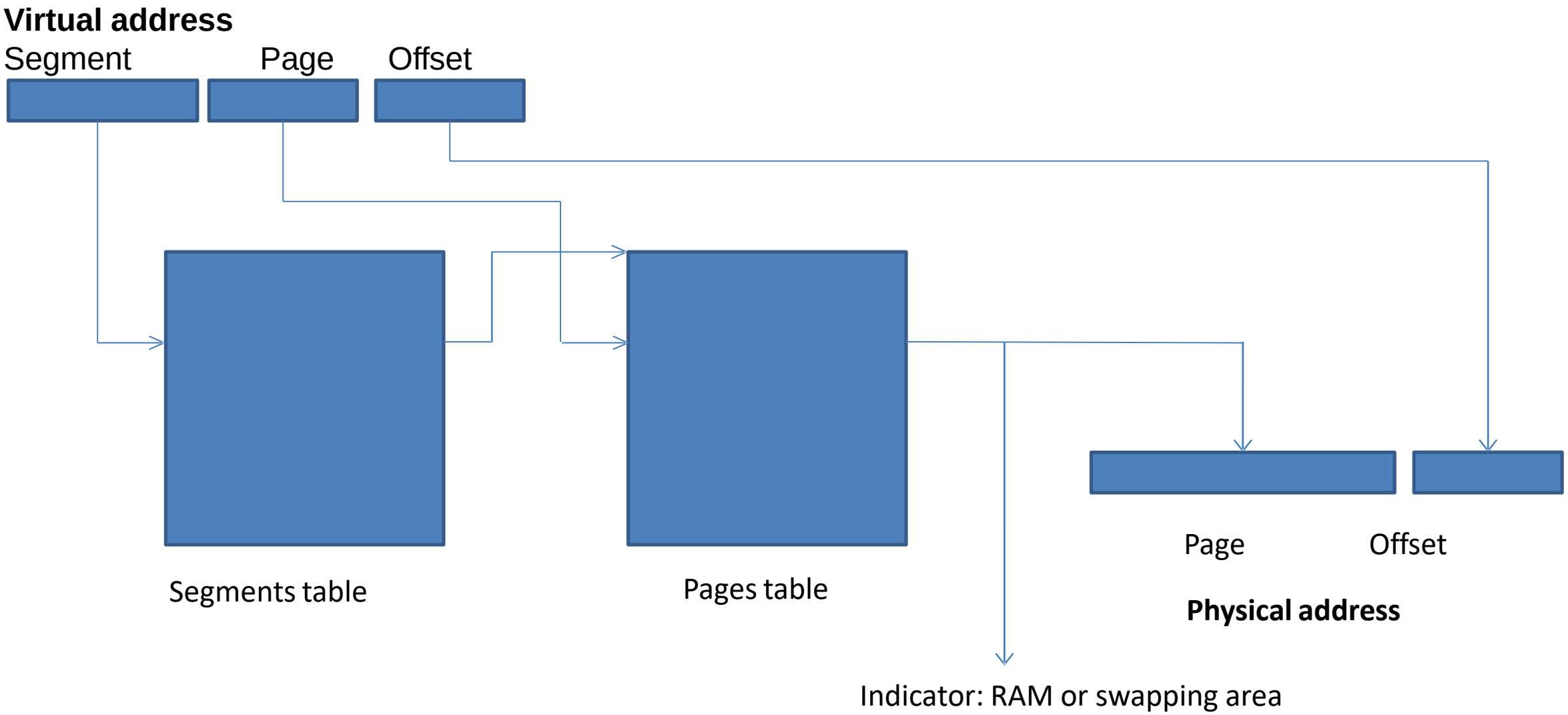
Modern computer memory structure

- **Segment-page memory**
- Page – constant size, measured in bytes or words
- Segment – variable size, measured in pages
- It is required that only pages, containing running parts of code and their data, are in RAM, other pages – in swapping area of HD
- Thus, the process size can exceed RAM size
- Pages can be scattered in memory – non-contiguous memory of a process; special tables provide memory mapping
- Hardware support – segment and page registers of CPU

Virtual Memory Concept



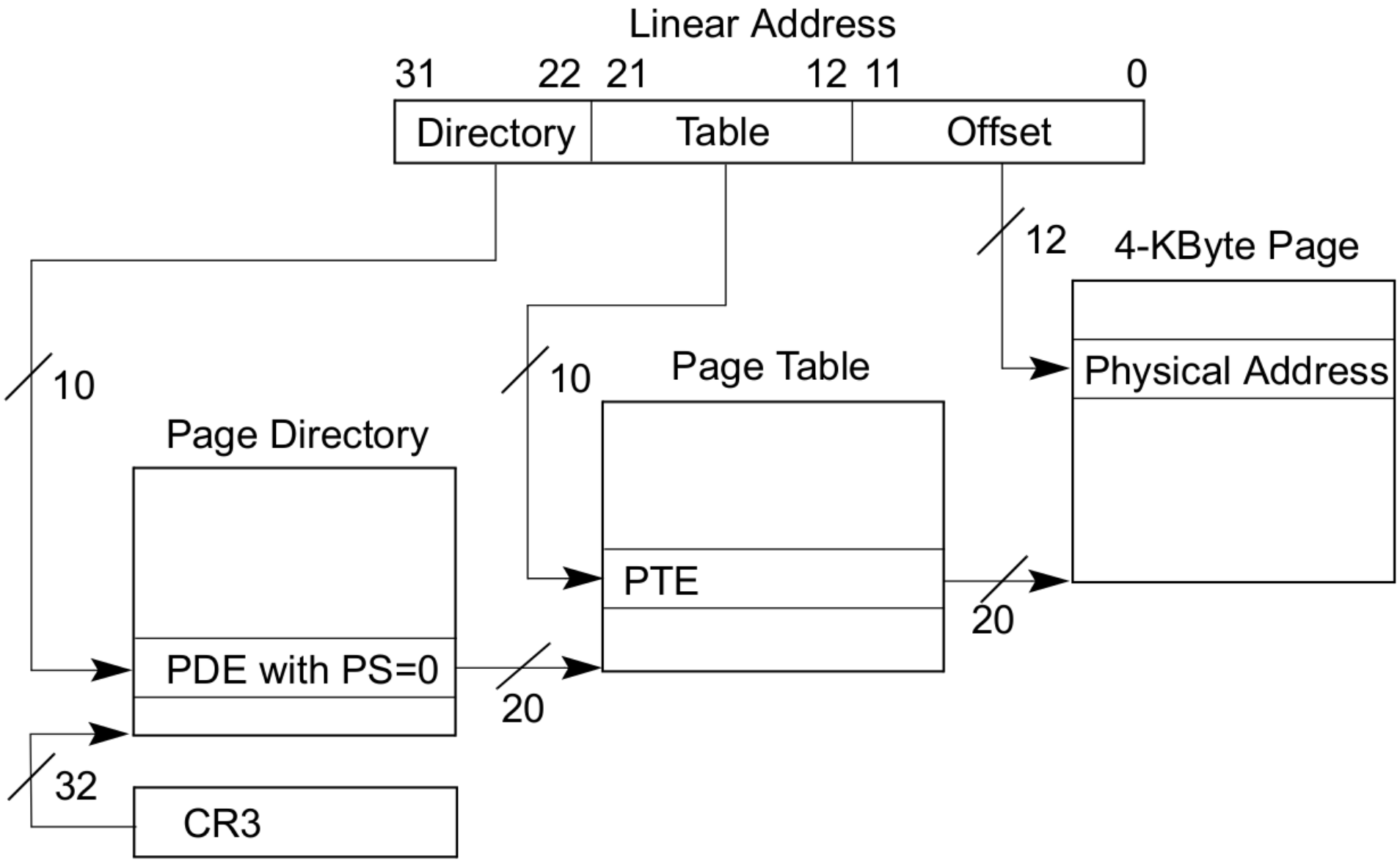
Segments and pages



Principles of page swapping

- Only a few pages of a task are pre-loaded; one containing the entry point
- Other pages are stored in the swapping area of disc
- A page fault occurs when a program tries to access a page not currently in RAM – an interrupt to enter kernel
- If there is a free page of RAM, the absent page is loaded, page tables updated, otherwise – page replacement (swapping)
- Page replacement algorithms: Optimal (theoretical best); FIFO (First-In, First-Out); LRU (Least Recently Used); Clock (second-chance algorithm)
- Thrashing happens when too many page faults occur.

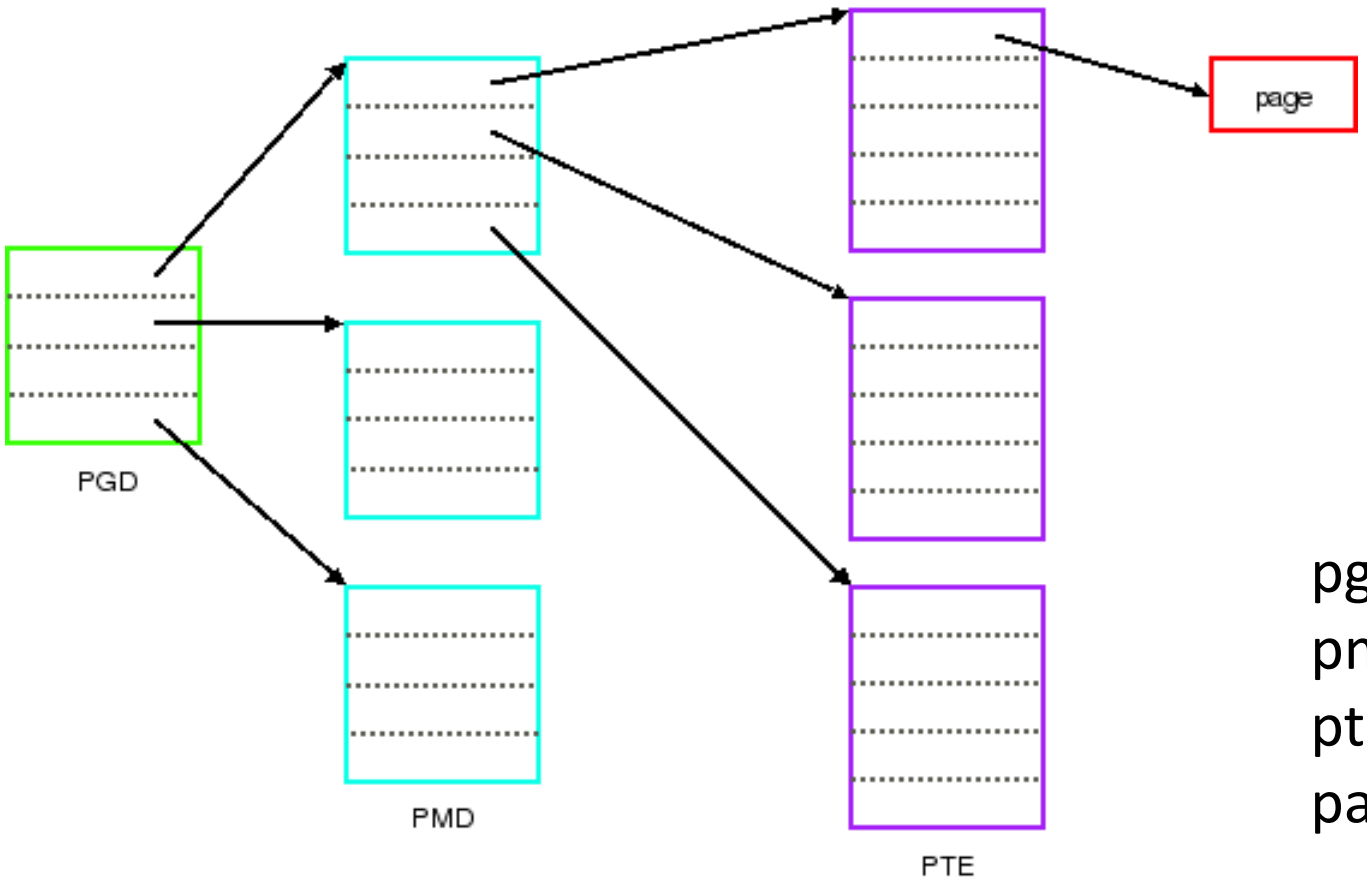
Segment-page memory in X86_32 architecture



Formats of CR3 and Paging-Structure Entries

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Address of page directory[1]																				Ignored							PCD	PWT	Ignored			**CR3**
Bits 31:22 of address of 4MB page frame										Reserved (must be 0)					Bits 39:32 of address[2]				PAT	Ignored			G	1	D	A	PCD	PWT	U/S	R/W	1	**PDE: 4MB page**
Address of page table																				Ignored				0	Ign	A	PCD	PWT	U/S	R/W	1	**PDE: page table**
Ignored																															0	**PDE: not present**
Address of 4KB page frame																				Ignored			G	PAT	D	A	PCD	PWT	U/S	R/W	1	**PTE: 4KB page**
Ignored																															0	**PTE: not present**

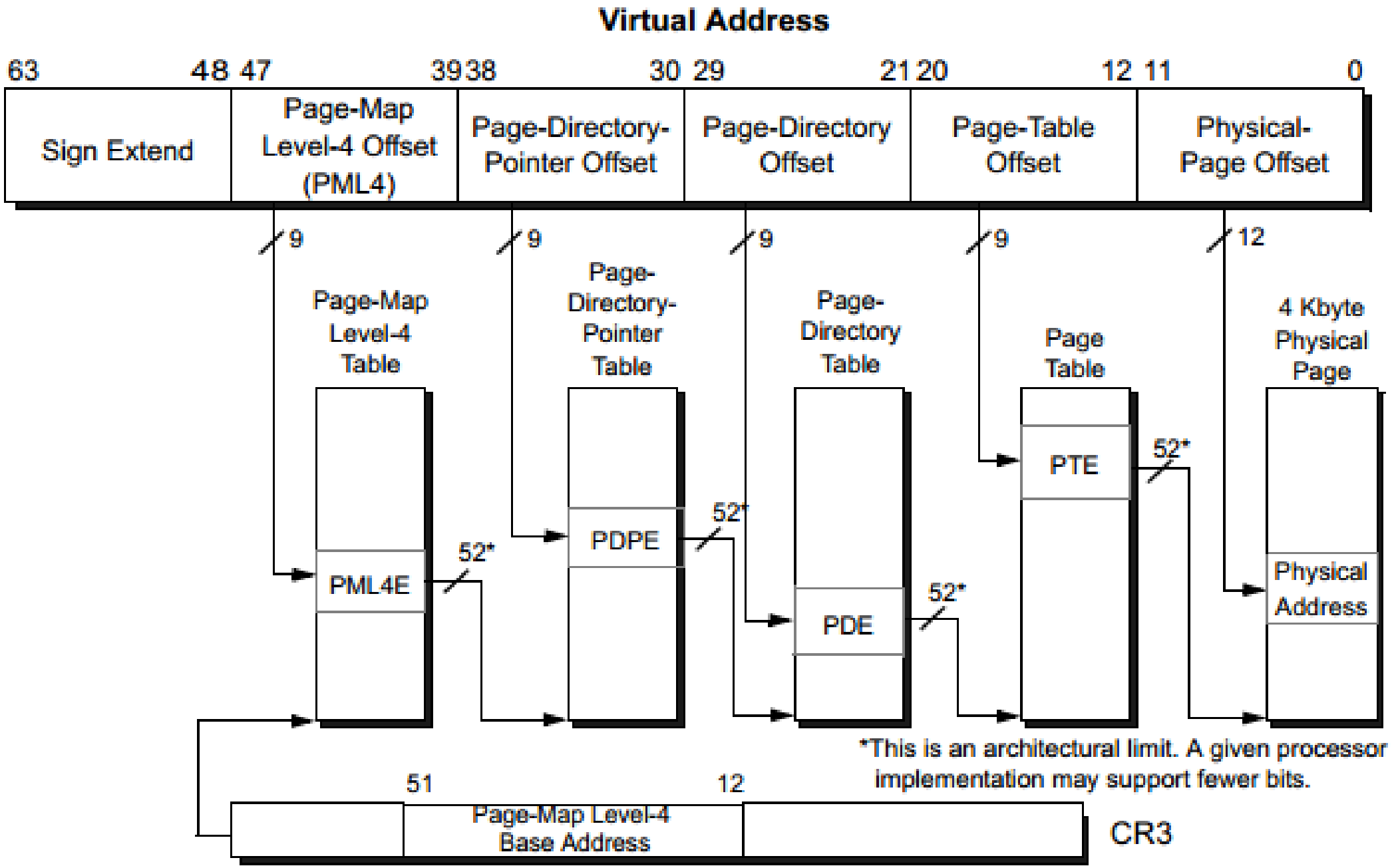
Actual 4-level structure of X86_64



The page global directory (PGD) contains only four entries, each corresponding to 1GB of virtual address space; the PGD is indexed using the top two bits of the virtual address. Each PGD entry points to a page middle directory (PMD), which holds 512 entries indexed by bits 21-29 of the virtual address.
The PMD entry (if it is not empty) points to an actual page table.

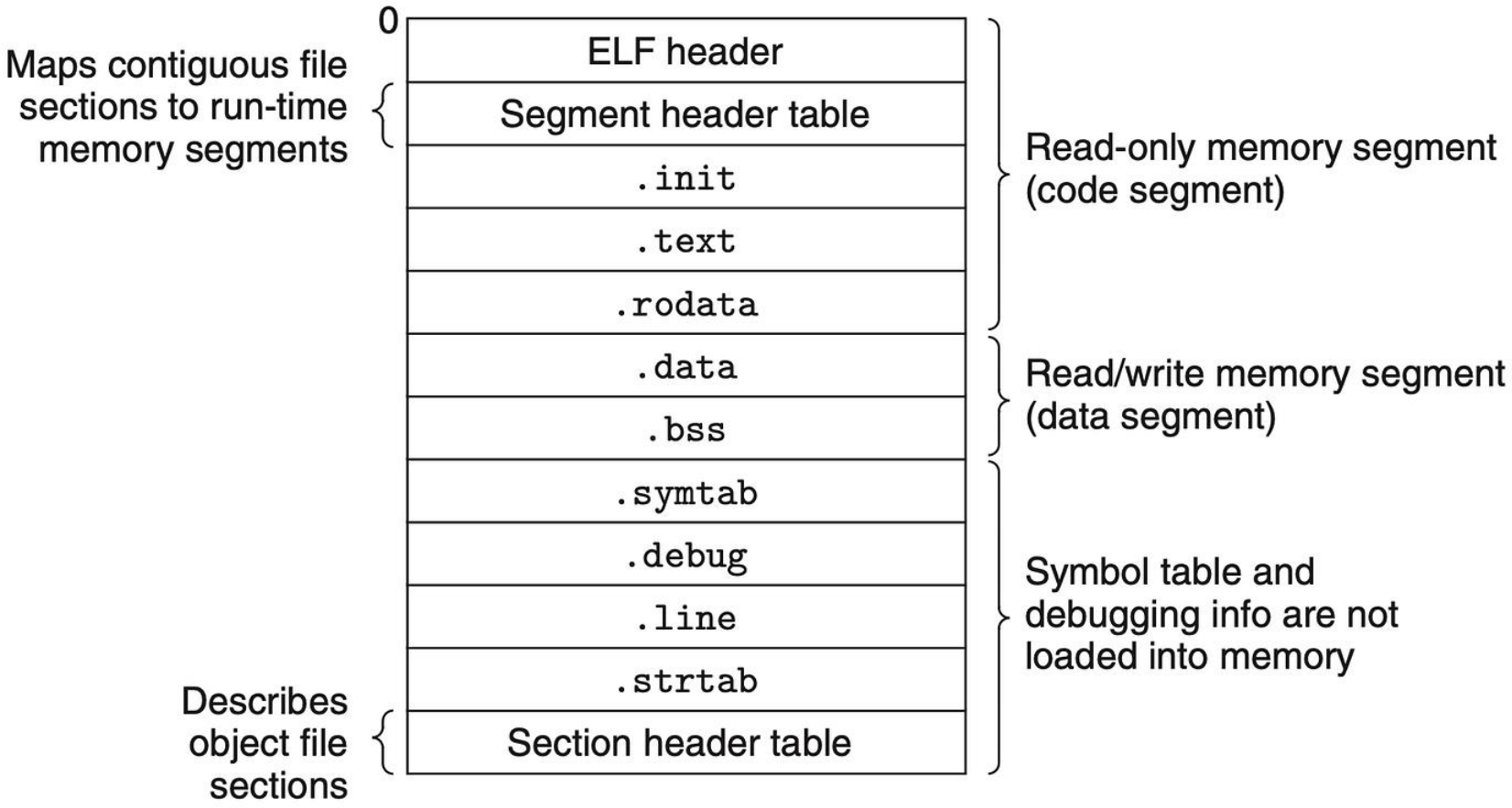
```
pgd = pgd_offset(address);
pmd = pmd_offset(pgd, address);
pte = *pte_offset_map(pmd, address);
page = pte_page(pte);
```

x86-64 paging architecture



Structure of executable file

- Header
- Segment(s) of code
- Segment(s) of data
- Stack
- Heap
- Shared segments



Types of memory in C program

- Static – declaration outside functions
- External – a reference
- Stack – declaration inside a function – created when entering the function and destroyed when exiting
- Dynamic – implicit allocation/deallocation with malloc/free – allocated within heap (dynamic partition technique)

Allocate and free dynamic memory

- Allocate

```
void *malloc(size_t size);
```

- Free

```
void free(void *ptr);
```

- Allocate array

```
void *calloc(size_t nmemb, size_t size);
```

- Reallocate

```
void *realloc(void *ptr, size_t size);
```

Implementation of dynamic memory

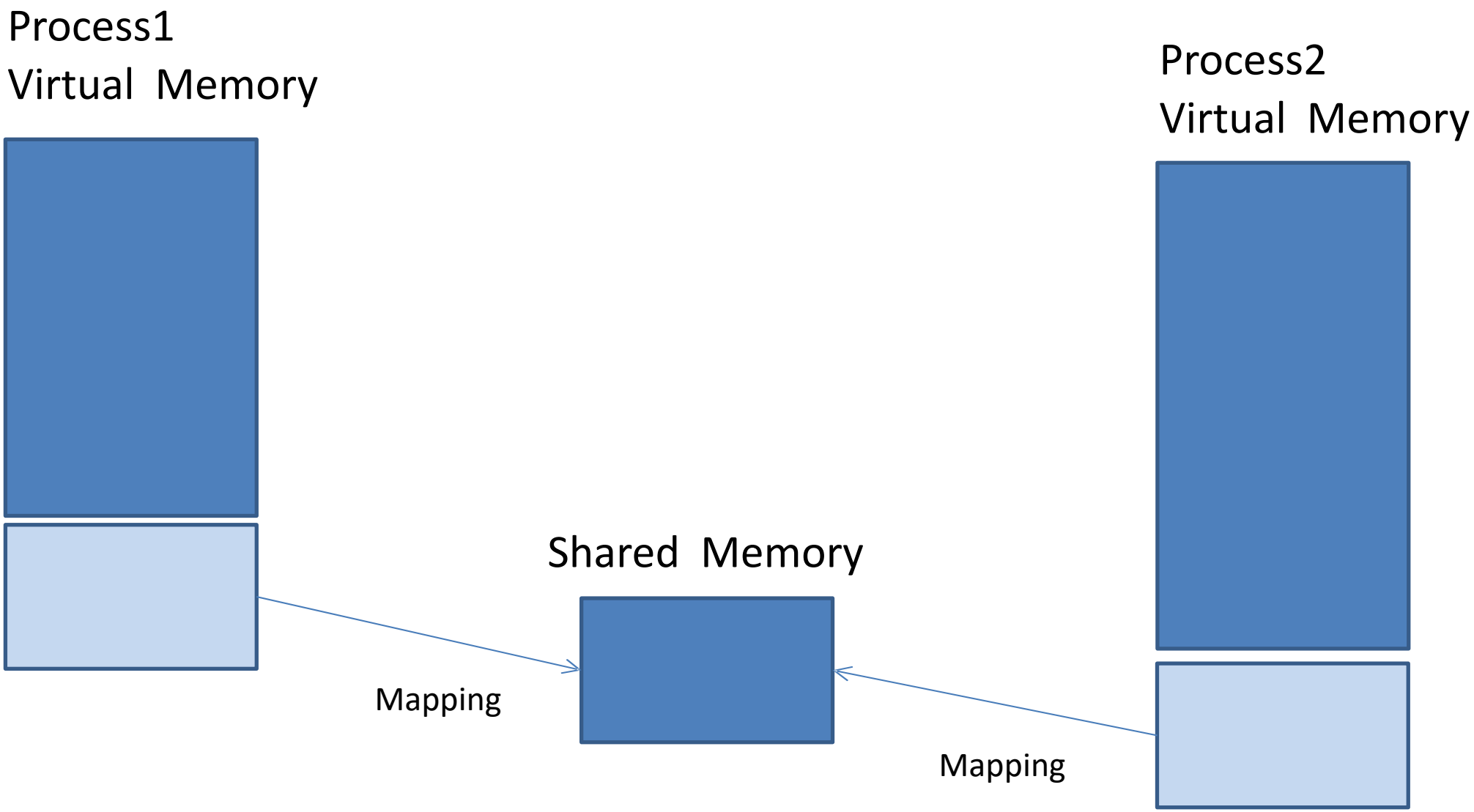
Free sections
of heap

Allocated
sections of heap

Section descriptor: address, length, attributes

Descriptor location – before a section

Shared memory concept



Shared memory segment

```
#include <sys/types.h>
#include <sys/shm.h>
```

- Allocate a segment

```
int shmget(key_t key, size_t size, int shmflg);
```

- Attach a shared memory segment

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

- Detach the shared memory segment

```
int shmdt(const void *shmaddr);
```

POSIX shared memory functions

- #include <sys/mman.h>
- Create/open shared memory object

```
int shm_open(const char *name, int oflag, mode_t mode);
```

- Unlink shared memory object

```
int shm_unlink(const char *name);
```

POSIX shared memory functions (continuation)

- truncate a file to a specified length

```
int truncate(const char *path, off_t length);
int ftruncate(int fd, off_t length);
```

- map files or devices into memory

```
void *mmap(void *addr, size_t length, int prot, int flags,
                    int fd, off_t offset);
```

- unmap files or devices from memory

```
int munmap(void *addr, size_t length);
```

Example 1
Server

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#define SHMSZ 27
int main()
{
   char c;
   int shmid;  key_t key;  char *shm, *s;  key = 5678;
   if ((shmid = shmget(key, SHMSZ, IPC_CREAT | 0666)) < 0) {
     perror("shmget");
     exit(1);
   }
   if ((shm = shmat(shmid, NULL, 0)) == (char *) -1) {
     perror("shmat");
     exit(1);
   }
   *s = shm;
   for (c = 'a'; c <= 'z'; c++)
         *s++ = c;
   *s = '\0';
   while (*shm != '*')
         sleep(1);
   exit(0);
}
```

Example 1 Client

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#define SHMSZ 27
int main()
{
   int shmid;  key_t key;  char *shm,
   *s; key = 5678;
   if ((shmid = shmget(key, SHMSZ, 0666)) < 0) {
     perror("shmget");  exit(1); }
   if ((shm = shmat(shmid, NULL, 0)) == (char *) -1) {
     perror("shmat"); exit(1); }
  for (s = shm; *s != NULL; s++)  putchar(*s);
  putchar('\n');
  *shm = '*';
   exit(0);
}
```

Example 2

```
#include <semaphore.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/mman.h>
int main(int argc, char **argv)
{
 int fd, i,count=0,nloop=10,zero=0,*ptr; sem_t
 mutex;
 fd = open("log.txt",O_RDWR|O_CREAT,S_IRWXU);
 write(fd,&zero,sizeof(int));
 ptr = mmap(NULL,sizeof(int),
  PROT_READ |PROT_WRITE,MAP_SHARED,fd,0);
 close(fd);
 if( sem_init(&mutex,1,1) < 0) {
    perror("semaphore initilization"); exit(0);
    }
```

```
if (fork() == 0) { /* child process*/
   for (i = 0; i < nloop; i++) {
      sem_wait(&mutex);
      printf("child: %d\n", (*ptr)++);
      sem_post(&mutex);
    }
    exit(0);
}
 /* parent process */
 for (i = 0; i < nloop; i++) {
     sem_wait(&mutex);
     printf("parent: %d\n", (*ptr)++);
     sem_post(&mutex);
 }
 exit(0);
}
```

POSIX shared memory & semaphores